

CODING TECHNICAL ASSESSMENT

Position: Full Stack Java Developer

Level: Mid-level

Technologies: Java, HTML, CSS, JavaScript

INSTRUCTIONS:

- This is a coding-focused technical assessment
- Complete all coding exercises and projects
- Submit your code files with clear documentation
- Time limit: 2-3 hours
- Reply to this email with your completed solutions

****Full Stack Java Developer Mid-Level Coding Assessment****

****Time Limit:** 2 hours 30 minutes**

****Instructions:****

1. Please create a new folder named `full-stack-java-developer` and create separate folders for each section (coding-exercises, practical-projects, code-review-tasks).
2. Use your preferred IDE or text editor to write the code.
3. For coding exercises, please submit the code in a file with the same name as the exercise (e.g., `exercise1.java`).
4. For practical projects, please submit the entire project folder (e.g., `project1`).
5. For code review tasks, please submit the refactored code in a file with the same name as the original code (e.g., `review1.java`).
6. Submit your code as a zip file named `full-stack-java-developer.zip`.

****Section 1: CODING EXERCISES (6 questions)****

****Exercise 1: Algorithm - Binary Search****

* Problem Statement: Implement a binary search algorithm to find an element in a sorted array.

* Input/Output Examples:

'2 -ç WC `arr = [1, 2, 3, 4, 5, 6, 7, 8, 9]`, `target = 5`

'2 ÷WG WC `4` (index of the target element)

* Expected Time to Complete: 15 minutes

* Evaluation Criteria:

'2 6÷' &V7FæW72 öb F†R Æv÷ ithm

'2 Vff-6-Væ7' öb F†R Æv÷ ithm (time complexity)

'2 6öFR V Æ—G' æB &V F &-Æ—G•

* Sample Test Cases:

'2 "" Ò ³ Â "Â 2Â BÂ RÂ bÂ rÂ ,Â •Ö Â F &vWB Ò V Óâ F

'2 "" Ò ³ Â "Â 2Â BÂ RÂ bÂ rÂ ,Â •Ö Â F &vWB Ò Óâ Ó †æ÷B `ound)

Exercise 2: Data Structure - Stack

* Problem Statement: Implement a simple Stack using an array.

* Input/Output Examples:

'2 -ç WC `push(1)`, `push(2)`, `pop()`

'2 ÷WG WC `2`

* Expected Time to Complete: 20 minutes

* Evaluation Criteria:

'2 6÷' &V7FæW72 öb F†R 7F 0k implementation

'2 Vff-6-Væ7' öb F†R 7F 0k operations (time complexity)

'2 6öFR V Æ—G' æB &V F &-Æ—G•

* Sample Test Cases:

'2 W6,f - Â W6,f"- Â ÷ ,- Óâ &

'2 W6,f - Â W6,f"- Â W6,f2- Â ÷ ,- Óâ 6

Exercise 3: Code Implementation - Calculator

* Problem Statement: Implement a simple calculator that takes in a string of mathematical operations and evaluates the result.

* Input/Output Examples:

'2 -ç WC `"2+2\*3"`

'2 ÷WG WC `8`

* Expected Time to Complete: 25 minutes

* Evaluation Criteria:

'2 6÷' &V7FæW72 öb F†R 6 Æ7VÆ F÷" -× ÆVÖVçF F-öà

'2 Vff-6-Væ7' öb F†R 6 Æ7VÆ F÷" ‡F-ÖR 6ö× ÆPxity)

'2 6öFR V Æ—G' æB &V F &-Æ—G•

* Sample Test Cases:

'2 #"3"£2& Óâ †

'2 # ó"Ó & Óâ F

Exercise 4: Bug Fixing - Reverse String

* Problem Statement: Fix the bug in the given code snippet that is supposed to reverse a string.

* Input/Output Examples:

'2 -ç WC `"hello"

'2 ÷WG WC `"olleh"

* Expected Time to Complete: 15 minutes

* Evaluation Criteria:

'2 6÷' &V7FæW72 öb F†R ug fix

'2 6öFR V Æ—G' æB &V F &-Æ—G•

* Sample Test Cases:

'2 &†VÆÆð& Óâ &öÆÆV,&

'2 'porld"" => `"dlrow"

Exercise 5: Code Optimization - Fibonacci

* Problem Statement: Optimize the given code snippet that calculates the Fibonacci sequence.

* Input/Output Examples:

'2 -ç WC `n = 10`

'2 ÷WG WC `55`

* Expected Time to Complete: 20 minutes

* Evaluation Criteria:

'2 Vff-6-Væ7' öb F†R ÷ F-Ö— ed code (time complexity)

'2 6öFR V Æ—G' æB &V F &—Æ—G•

* Sample Test Cases:

'2 â Ò Óâ SV

'2 â Ò # Óâ cscV

****Exercise 6: API Development - Users API****

* Problem Statement: Implement a simple REST API using Java that exposes two endpoints: `/users` and `/users/{id}`.

* Input/Output Examples:

'2 —ç WC `GET /users`

'2 ÷WG WC `[{"id": 1, "name": "John"}, {"id": 2, "name": "Jane"}]`

* Expected Time to Complete: 30 minutes

* Evaluation Criteria:

'2 6÷' &V7FæW72 öb F†R ' —× ÆVÖVçF F—öà

'2 Vff—6—Væ7' öb F†R ' ‡F—ÖR 6ö× ÆPxity)

'2 6öFR V Æ—G' æB &V F &—Æ—G•

* Sample Test Cases:

'2 tUB ÷W6W'6 Óâ ·2&—B# 1, "name": "John"}, {"id": 2, "name": "Jane"}]`

'2 tUB ÷W6W'2ó Óâ ·2&—B# 1, "name": "John"}`

****Section 2: PRACTICAL PROJECTS (2 mini-projects)****

****Project 1: Todo List App****

* Problem Statement: Build a simple Todo List App using Java, HTML, CSS, and JavaScript.

* Requirements:

'2 W6W" 6 â FB æPw tasks

'2 W6W" 6 â FVÆWFR Pxisting tasks

'2 W6W" 6 â Ö k tasks as completed

* Expected Time to Complete: 45 minutes

* Evaluation Criteria:

'2 6÷' &V7FæW72 öb F†R —× ÆVÖVçF F—öà

'2 Vff-6-Væ7' öb F†R ‡F-ÖR 6ö× ÆPxity)

'2 6öFR V Æ—G' æB &V F &-Æ—G•

* Sample Test Cases:

'2 FB æPw task => Task is added to the list

'2 FVÆWFR Pxisting task => Task is removed from the list

'2 Ö k task as completed => Task is marked as completed

****Project 2: Weather API Client****

* Problem Statement: Build a simple Weather API Client using Java that displays the current weather for a given city.

* Requirements:

'2 W6W" 6 â -ç WB 6—G' æ ÖP

'2 F—7 Æ ys current weather for the city

'2 F—7 Æ ys error message if city not found

* Expected Time to Complete: 45 minutes

* Evaluation Criteria:

'2 6÷' &V7FæW72 öb F†R -× ÆVÖVçF F-öà

'2 Vff-6-Væ7' öb F†R ‡F-ÖR 6ö× ÆPxity)

'2 6öFR V Æ—G' æB &V F &-Æ—G•

* Sample Test Cases:

'2 -ç WB 6—G' æ ÖR Óâ F—7 Æ ys current weather

'2 -ç WB -àvalid city name => App displays error message

****Section 3: CODE REVIEW TASKS (2 questions)****

****Review 1: Code Refactoring - Calculator****

* Problem Statement: Refactor the given code snippet to improve code quality and readability.

* Input/Output Examples:

'2 -ç WC `calculateArea(10, 20)`

'2 ÷WG WC `200`

* Expected Time to Complete: 20 minutes

* Evaluation Criteria:

'2 6÷' &V7FæW72 öb F†R &V`actored code

'2 6öFR V Æ—G' æB &V F &—Æ—G•

* Sample Test Cases:

'2 6 Æ7VÆ FT &V f Â# – Óâ #

'2 6 Æ7VÆ FT &V fRÂ – Óâ S

****Review 2: Bug Fixing - String Concatenation****

* Problem Statement: Identify and fix the bug in the given code snippet that is supposed to concatenate two strings.

* Input/Output Examples:

'2 –ç WC `"hello"` and `"world"`

'2 ÷WG WC `"helloworld"`

* Expected Time to Complete: 20 minutes

* Evaluation Criteria:

'2 6÷' &V7FæW72 öb F†R ug fix

'2 6öFR V Æ—G' æB &V F &—Æ—G•

* Sample Test Cases:

'2 &†VÆÆò& æB 'porld'" => `"helloworld"`

'2 &†va"" and `"developer"` => `"javadeveloper"`

****Submission Guidelines:****

1. Please submit your code as a zip file named `full-stack-java-developer.zip`.
2. Make sure to include all the required files and folders in the zip file.
3. Please follow the file naming conventions and folder structure as specified above.
4. Please test your code thoroughly before submitting it.
5. Please include a README file with instructions on how to run and test your code.

****Evaluation Criteria:****

1. Correctness of the code
2. Efficiency of the code (time complexity)

3. Code quality and readability
4. Adherence to the problem statement and requirements
5. Ability to debug and fix errors

****Note:****

1. Please use Java 8 or later for all coding exercises and projects.
2. Please use a Java IDE or text editor of your choice.
3. Please use HTML, CSS, and JavaScript for the Todo List App and Weather API Client projects.
4. Please use a Weather API of your choice for the Weather API Client project.
5. Please make sure to handle errors and exceptions properly in your code.

Generated on: 15/09/2025 14:33:18